

THE PROMPT LADDER

Prompt iteration document

Name	Cleidyanne Castro Pereira
Track	Backend AI Engineering
Program	FlyRank AI Internship
Assignment	Week 2 - General AI Fluency

One weak baseline + five controlled iterations

1. Assignment Overview

This document demonstrates how a weak prompt was improved through five controlled iterations. Each version adds exactly one named layer, and every output is compared with the previous one.

Selected task

Review a FastAPI endpoint from a beginner task-management CRUD API and identify improvements without overengineering the project.

Code used for all six runs

FASTAPI CODE

```
@app.get("/tasks/{task_id}")
def get_task(task_id: int):
    for task in tasks:
        if task["id"] == task_id:
            return task

    return {"error": "Task not found"}
```

Expected behavior

- Return the matching task when the ID exists.
- Return HTTP 404 with {"error": "Task not found"} when the ID does not exist.
- Keep the implementation appropriate for a beginner in-memory CRUD assignment.

Run 1 - Weak Baseline

Named layer: Baseline - no additional layer

PROMPT

Review this FastAPI code.


```
@app.get("/tasks/{task_id}")
def get_task(task_id: int):
    for task in tasks:
        if task["id"] == task_id:
            return task

    return {"error": "Task not found"}
```

REPRESENTATIVE OUTPUT

The code loops through the tasks list and returns the task matching the requested ID. It could be improved by adding documentation, type hints, exception handling, a database, authentication, logging, tests, and a service layer.

Comparison notes

What changed	Nothing. This was the intentionally weak baseline.
What improved	Not applicable because this was the first output.
What still failed	The response was generic and suggested many unrelated production features. It did not identify the incorrect 200 response for a missing task.
What I would try next	State a clearer goal so the model focuses on bugs, API behavior, and maintainability.

Run 2 - Clearer Goal

Named layer: Clearer goal

PROMPT
Review this FastAPI code and identify bugs, maintainability issues, and improvements that would make it more production-ready. <pre>@app.get("/tasks/{task_id}") def get_task(task_id: int): for task in tasks: if task["id"] == task_id: return task return {"error": "Task not found"}</pre>
REPRESENTATIVE OUTPUT
The main functional issue is that a missing task returns a JSON error with HTTP 200. The route should raise <code>HTTPException(status_code=404, detail='Task not found')</code>. Additional production improvements include database persistence, structured logging, dependency injection, authentication, and automated tests.

Comparison notes

What changed	I specified the kinds of issues the model should identify.
What improved	The model found the most important bug: the route returns 200 instead of 404 for an unknown ID.
What still failed	It still proposed several advanced production features that were unnecessary for the assignment.
What I would try next	Define the audience so the explanation matches a junior backend developer's level.

Run 3 - Defined Audience

Named layer: Defined audience

PROMPT

Review this FastAPI code and identify bugs, maintainability issues, and improvements that would make it more production-ready.

Explain the findings for a junior backend developer who understands basic Python and FastAPI.

```
@app.get("/tasks/{task_id}")
def get_task(task_id: int):
    for task in tasks:
        if task["id"] == task_id:
            return task

    return {"error": "Task not found"}
```

REPRESENTATIVE OUTPUT

Your loop is valid for a small list, but the missing-task path is the main bug. Returning a dictionary does not change the HTTP status code, so the client receives 200 OK. In FastAPI, raise HTTPException with status code 404. For learning purposes, this is the first change to make before considering databases or more advanced architecture.

Comparison notes

What changed	I defined the audience as a junior backend developer.
What improved	The explanation became easier to understand and clarified why returning a dictionary does not produce a 404 response.
What still failed	The answer became longer and more tutorial-like than necessary. This change improved clarity but reduced concision.
What I would try next	Provide the real project context so the recommendations stay proportional to the assignment.

Run 4 - Real Context

Named layer: Real context

PROMPT

Review this FastAPI code and identify bugs, maintainability issues, and improvements that would make it more production-ready.

Explain the findings for a junior backend developer who understands basic Python and FastAPI.

This endpoint belongs to a beginner CRUD API for managing tasks. The current version stores tasks in memory and is intended to demonstrate API fundamentals rather than production deployment.

```
@app.get("/tasks/{task_id}")
def get_task(task_id: int):
    for task in tasks:
        if task["id"] == task_id:
            return task

    return {"error": "Task not found"}
```

REPRESENTATIVE OUTPUT

For this assignment, keeping the in-memory list and the loop is reasonable. The essential correction is the missing-task response: raise `HTTPException(status_code=404, detail='Task not found')`. A small readability improvement would be to name the route function `get_task_by_id`. A database, authentication, and a service layer are outside the current learning goal.

Comparison notes

What changed	I explained that the code belongs to a beginner in-memory CRUD API.
What improved	The model stopped recommending unrelated infrastructure and focused on the endpoint's actual learning goal.
What still failed	The response was relevant but presented as one paragraph, making the findings harder to scan.
What I would try next	Specify a structured output format.

Run 5 - Output Format

Named layer: Specified output format

PROMPT

Review this FastAPI code and identify bugs, maintainability issues, and improvements that would make it more production-ready.

Explain the findings for a junior backend developer who understands basic Python and FastAPI.

This endpoint belongs to a beginner CRUD API for managing tasks. The current version stores tasks in memory and is intended to demonstrate API fundamentals rather than production deployment.

Return the review using these sections:

1. What the code does
2. What is already correct
3. Bugs or risks
4. Recommended improvements
5. Revised code

```
@app.get("/tasks/{task_id}")
def get_task(task_id: int):
    for task in tasks:
        if task["id"] == task_id:
            return task

    return {"error": "Task not found"}
```

REPRESENTATIVE OUTPUT

1. What the code does: Searches the in-memory task list for a matching ID.
2. What is already correct: The path parameter is typed and the search logic works for a small list.
3. Bugs or risks: A missing task returns HTTP 200 instead of 404.
4. Recommended improvements: Raise HTTPException and use a more descriptive function name.
5. Revised code:

```
@app.get('/tasks/{task_id}')
def get_task_by_id(task_id: int):
    for task in tasks:
        if task['id'] == task_id:
            return task
    raise HTTPException(status_code=404, detail="Task not found")
```

Comparison notes

What changed	I requested five explicit sections.
What improved	The output became easy to scan and clearly separated correct behavior, risks, and revised code.
What still failed	The revised code changed the error body from {'error': 'Task not found'} to FastAPI's default {'detail': 'Task not found'}, which may violate the assignment's required response format.
What I would try next	Add verification requirements so the model preserves the exact API contract.

Run 6 - Verification Requirements

Named layer: Verification requirements

PROMPT

Review this FastAPI code and identify bugs, maintainability issues, and improvements that would make it more production-ready.

Explain the findings for a junior backend developer who understands basic Python and FastAPI.

This endpoint belongs to a beginner CRUD API for managing tasks. The current version stores tasks in memory and is intended to demonstrate API fundamentals rather than production deployment.

Return the review using these sections:

1. What the code does
2. What is already correct
3. Bugs or risks
4. Recommended improvements
5. Revised code

Before providing the final answer, verify that the revised code preserves the existing endpoint path, request model, response behavior, validation rules, and HTTP status codes. Do not introduce libraries that are unnecessary for this small project.

Required behavior: unknown IDs must return HTTP 404 with {'error': 'Task not found'}.

```
@app.get("/tasks/{task_id}")
def get_task(task_id: int):
    for task in tasks:
        if task["id"] == task_id:
            return task

    return {"error": "Task not found"}
```

REPRESENTATIVE OUTPUT

1. What the code does: Searches an in-memory list and returns the task whose ID matches the path parameter.
2. What is already correct: The route, typed parameter, and search logic are suitable for this beginner project.
3. Bugs or risks: Returning a dictionary for a missing task produces HTTP 200.
4. Recommended improvements: Return a JSONResponse with status code 404 so both the status and required body are preserved.
5. Revised code:

```
@app.get('/tasks/{task_id}')
def get_task(task_id: int):
    for task in tasks:
        if task['id'] == task_id:
            return task
    return JSONResponse(status_code=404, content={'error': 'Task not found'})
```

Verification: The endpoint path, successful response behavior, in-memory storage, and exact 404 JSON body remain unchanged.

Comparison notes

What changed	I added explicit verification requirements and the exact missing-task response contract.
What improved	The model preserved both the HTTP 404 status and the required {'error': 'Task not found'} body instead of silently changing it to FastAPI's default detail format.
What still failed	The answer repeated part of the same explanation in both the recommendation and verification sections.
What I would try next	Add a concision constraint, such as avoiding repeated explanations or limiting the review to 350 words.

8. Final Reusable Prompt

The final version below is written as a template so another backend learner can use it without knowing the personal context behind this assignment.

FINAL TEMPLATE

Act as an experienced [LANGUAGE/Framework] backend engineer.

Review the code below and identify:

- Functional bugs
- Validation and error-handling problems
- Framework convention issues
- Maintainability concerns
- Improvements appropriate to the size and purpose of the project

Audience:

Explain the findings for a [EXPERIENCE LEVEL] developer who understands [EXISTING KNOWLEDGE].

Project context:

This code belongs to [PROJECT TYPE]. Its purpose is to [PURPOSE]. It currently uses [CURRENT ARCHITECTURE OR STORAGE]. Keep recommendations proportionate to the project and avoid unnecessary complexity.

Return the review using these sections:

1. What the code currently does
2. What is already correct
3. Bugs or risks
4. Recommended improvements
5. Revised code
6. Final verification

Before returning the answer, verify that the revised code preserves:

- Existing endpoint paths
- Request and response behavior
- Validation rules
- Required HTTP status codes
- Any other requirements listed below

Required behavior:

[PASTE REQUIREMENTS]

Do not introduce unnecessary libraries or architecture.

Code:

[PASTE CODE]

9. Reflection

The largest improvement came from adding real project context and verification requirements. Context prevented the model from overengineering a beginner assignment, while verification prevented the revised code from changing the required JSON error format. Defining the audience made the explanation clearer, but it also made the response more verbose. This confirmed that prompt improvements are not always purely positive; each layer introduces a trade-off that must be evaluated against the task.

Submission checklist

- ✓ Six runs total: one baseline plus five iterations
- ✓ Exactly one named layer added per iteration
- ✓ Representative output included for every run
- ✓ Four comparison notes included for every run
- ✓ At least one honest example of a change making the output worse
- ✓ Final prompt reusable by another person